

CTFL 4.0

Curso Completo de Certificação

ISTQB® Certified Tester — Foundation Level

Do zero à aprovação

Teoria explicada · Analogias · Exemplos · Dicas de prova

Baseado no Syllabus oficial v4.0 (BSTQB)

Sumário

Bem-vindo: como usar este curso

Este material foi escrito com um objetivo único e claro: levar você, mesmo que nunca tenha estudado teoria de teste de software, a entender de verdade os conceitos do syllabus CTFL 4.0 e passar na prova de certificação do ISTQB.

Você já testa software na prática. Isso é uma vantagem enorme, mas também esconde uma armadilha: a prova não pergunta "o que você faria", e sim "o que o syllabus define". Muitas vezes o nome oficial de algo que você já faz é diferente do nome que você usa no dia a dia. Por isso, em cada tópico eu vou conectar a teoria àquilo que você já vive no trabalho, dar uma analogia para fixar, mostrar um exemplo concreto e destacar exatamente o que costuma cair na prova.

A estrutura da prova CTFL 4.0

Antes de tudo, entenda o jogo que você vai jogar:

- São 40 questões de múltipla escolha.
- Cada questão vale 1 ponto. Você precisa de 65% para passar, ou seja, 26 acertos.
- O tempo é de 60 minutos (75 minutos se você fizer a prova em um idioma que não é a sua língua nativa).
- Não há desconto por erro: questão em branco e questão errada valem a mesma coisa. Portanto, NUNCA deixe questão em branco.
- O conteúdo é dividido em 6 capítulos, e cada capítulo tem um número fixo de questões, proporcional ao tempo de estudo definido no syllabus.

A distribuição de questões por capítulo é a seguinte (decore isto, ajuda a priorizar o estudo):

Capítulo	Tema	Questões
1	Fundamentos de Teste	8
2	Teste ao longo do Ciclo de Vida (SDLC)	6
3	Teste Estático	4
4	Análise e Modelagem de Teste	11
5	Gerenciamento das Atividades de Teste	9
6	Ferramentas de Teste	2
—	TOTAL	40

Cai na prova

O Capítulo 4 (Análise e Modelagem) sozinho vale 11 questões — mais de 1/4 da prova. Os capítulos 4 e 5 juntos valem 20 das 40 questões. Se o seu tempo de estudo for curto, é nesses dois capítulos que

you need to be very firm. Chapter 4 is still the only one with K3 level questions (application), which require calculation and reasoning — not just guessing.

Os níveis cognitivos: K1, K2 e K3

The syllabus marks each learning objective with a cognitive level. Understanding this tells you WHAT TYPE of question to expect:

- **K1 (Lembrar):** a questão pede que você reconheça ou recorde um termo ou definição. Ex.: "Qual destes é um princípio de teste?". É memorização pura.
- **K2 (Compreender):** a questão pede que você explique, compare, distinga ou classifique. Ex.: "Qual a diferença entre verificação e validação?". Exige entendimento, não só memória.
- **K3 (Aplicar):** a questão te dá uma situação e pede que você aplique uma técnica para chegar a um resultado. Ex.: "Quantos casos de teste são necessários para 100% de cobertura?". Exige praticar.

Os objetivos K3 estão quase todos no Capítulo 4 (técnicas caixa-preta e ATDD) e em parte do Capítulo 5 (estimativa, priorização, relatório de defeitos). São esses tópicos que você precisa treinar fazendo exercícios, não apenas lendo.



Analogia — A carteira de motorista

Think of CTFL as the theoretical exam for a driver's license. You can already drive very well (your practice), but to pass you need to know the names of the signs, the safety distance, and the formal rules. Knowing how to drive isn't enough — you need to speak the language of the manual. This course is your manual, translated for your reality.

Como estudar com este material

1. Leia cada capítulo entendendo, não decorando. As analogias existem para que o conceito 'gruda'.
2. Preste atenção redobrada nas caixas amarelas (Cai na prova) e roxas (Pegadinha) — elas concentram onde os candidatos erram.
3. Ao terminar um capítulo, faça as questões do simulado correspondente (arquivo separado) ANTES de seguir adiante.
4. No fim, refaça todos os simulados. Se acertar 80% ou mais com tranquilidade, você está pronto.

Capítulo 1 — Fundamentos de Teste

Peso na prova: 8 questões (20%). É a base de tudo. Termos definidos aqui aparecem nas perguntas dos outros capítulos, então errar aqui custa caro.

1.1 O que é teste?

Existe um equívoco comum, e a prova adora explorá-lo: muita gente acha que testar é apenas "rodar o software e ver se quebra". Isso é apenas uma parte (a execução de teste). O syllabus define teste de software de forma muito mais ampla:

Teste de software é um conjunto de atividades para descobrir defeitos e avaliar a qualidade dos artefatos de software.

Repare em duas palavras-chave dessa definição:

- **Descobrir defeitos:** encontrar o que está errado.
- **Avaliar a qualidade:** dar informação sobre o quão bom (ou ruim) o produto está.

E os artefatos que você testa têm um nome oficial: quando estão sendo testados, são chamados de objetos de teste. Um objeto de teste pode ser código, mas também pode ser um documento de requisitos, uma história de usuário ou um projeto (design). Você não testa só o que executa.

Teste dinâmico vs. teste estático

Essa é uma das distinções mais importantes do capítulo:

- **Teste dinâmico:** envolve EXECUTAR o software. É o teste que você roda.
- **Teste estático:** NÃO executa o software. Você examina o artefato olhando para ele — revisões e análise estática (ver Capítulo 3).

Verificação vs. Validação — a pegadinha clássica

Estas duas palavras parecem sinônimos no português do dia a dia, mas na prova têm significados precisos e diferentes:

	Verificação	Validação
Pergunta que responde	Estamos construindo o produto CORRETAMENTE?	Estamos construindo o produto CERTO?
Foco	Atende aos requisitos especificados?	Atende às necessidades reais do usuário?
Exemplo	O campo aceita 11 dígitos como o requisito mandou?	Esse campo de 11 dígitos é mesmo o que o usuário precisava?

Analogia — Construindo uma casa

Verificação é conferir se o pedreiro seguiu a planta à risca (a parede está no lugar que o desenho mandou?). Validação é o morador entrar na casa pronta e perceber se ela atende ao que ele realmente queria viver (a planta estava certa em primeiro lugar?). Dá para verificar tudo perfeitamente e mesmo assim falhar na validação — porque a planta estava errada.

Cai na prova

Verificação = requisitos especificados ("do produto certo"... não! cuidado). Memorize assim: Verificação confere contra a ESPECIFICAÇÃO. Validação confere contra a NECESSIDADE/uso real. Teste estático pode fazer as DUAS coisas (verificação e validação).

1.1.1 Objetivos do teste

O syllabus lista os objetivos típicos do teste. Você não precisa decorar a lista inteira na ordem, mas precisa reconhecê-los e entender que os objetivos VARIAM conforme o contexto. Os principais são:

- Avaliar produtos de trabalho (requisitos, histórias, projeto, código).
- Detectar falhas e defeitos.
- Garantir a cobertura necessária do objeto de teste.
- Reduzir o nível de risco de qualidade inadequada.
- Verificar se os requisitos especificados foram atendidos (verificação).
- Validar se o objeto está completo e funciona como o stakeholder espera (validação).
- Verificar conformidade com requisitos contratuais, legais e normativos.
- Fornecer informação aos stakeholders para que tomem decisões.
- Criar confiança na qualidade do objeto de teste.

Cai na prova

Frase-chave do syllabus: "Os objetivos dos testes podem variar dependendo do contexto". Contexto inclui: o produto sendo testado, o nível de teste, os riscos, o SDLC seguido e fatores de negócio (estrutura da empresa, concorrência, tempo de mercado). Se a questão disser que os objetivos são SEMPRE os mesmos, está errada.

1.1.2 Teste e depuração (debugging) — não confunda!

Teste e depuração são atividades DISTINTAS, feitas muitas vezes por pessoas diferentes (testador x desenvolvedor). Essa separação cai na prova.

Teste no caso dinâmico DESENCADEIA falhas causadas por defeitos. O testador encontra o sintoma.

Depuração é a atividade (do desenvolvedor) de ENCONTRAR a causa da falha, analisá-la e ELIMINÁ-la.

O processo de depuração no teste dinâmico tem 3 passos, nesta ordem:

1. Reproduzir a falha.
2. Diagnosticar (encontrar a causa-raiz).
3. Corrigir a causa.

Depois da correção, entram dois testes que você precisa conhecer bem (detalhados no Cap. 2):

- **Teste de confirmação (reteste):** confirma que aquele defeito específico foi realmente corrigido. De preferência feito pela mesma pessoa que achou o defeito.
- **Teste de regressão:** verifica se a correção não quebrou nada em OUTRAS partes do sistema.

Atenção / Pegadinha

No teste ESTÁTICO, a depuração é mais simples: como o defeito é encontrado diretamente (não há execução, não há falha), não é preciso reproduzir nem diagnosticar. Você só remove o defeito. Questões de prova exploram exatamente essa diferença entre depurar no estático e no dinâmico.

1.2 Por que os testes são necessários?

Software com defeito pode causar prejuízo financeiro, perda de tempo, dano à reputação e, em casos extremos, ferimentos ou morte. O teste, como forma de controle de qualidade, ajuda a atingir os objetivos acordados dentro das restrições de escopo, tempo, qualidade e orçamento.

1.2.1 Contribuições do teste para o sucesso

- **Detecta defeitos de forma econômica** — que depois são removidos pela depuração (que não é teste). Assim o teste contribui INDIRETAMENTE para mais qualidade.
- **Avalia a qualidade diretamente** em vários estágios, ajudando decisões como a de liberar (release).
- **Representa o usuário no projeto** — os testadores garantem que a visão do usuário seja considerada.
- **Atende a exigências** contratuais, legais ou regulatórias.

1.2.2 Teste e Garantia da Qualidade (QA) — distinção que cai muito

As pessoas usam "teste" e "QA" como sinônimos, mas NÃO são a mesma coisa. Esta é uma das pegadinhas mais frequentes da prova.

Conceito	O que é	Orientação	Foco
QC (Controle de Qualidade)	Abordagem CORRETIVA, orientada ao PRODUTO. Teste é uma forma de QC.	Produto	Corrigir / detectar defeitos
QA (Garantia da Qualidade)	Abordagem PREVENTIVA, orientada ao PROCESSO. Responsabilidade de TODOS.	Processo	Melhorar processos

Conceito	O que é	Orientação	Foco
			para evitar defeitos

Analogia — A cozinha do restaurante

QC (controle de qualidade) é o chef provando cada prato antes de servir e devolvendo o que está ruim — age sobre o produto, depois de pronto, para corrigir.

QA (garantia da qualidade) é definir a receita, treinar a equipe, padronizar a higiene e o tempo de cozimento — age sobre o processo, antes, para que os pratos já saiam bons. O teste é o chef provando: é QC.

Cai na prova

Decore esta cadeia: Teste é uma forma de QC. QC é corretivo e orientado a produto. QA é preventivo e orientado a processo. Os resultados de teste são usados tanto pelo QC (para corrigir defeitos) quanto pelo QA (como feedback sobre a performance dos processos).

1.2.3 Erro, Defeito, Falha e Causa-raiz — a cadeia que SEMPRE cai

Esses quatro termos têm relação de causa e efeito, e a prova adora misturá-los. Entenda a sequência:

Erro (engano / mistake) → Defeito (bug / fault) → Falha (failure)

- **ERRO:** uma ação humana equivocada. O ser humano comete erros (por pressa, cansaço, complexidade, falta de treino).
- **DEFEITO:** o resultado do erro no artefato (no código, num documento de requisitos, num script de teste). É a falha estática esperando para acontecer.
- **FALHA:** quando o defeito é EXECUTADO e o sistema se comporta de forma errada. É o sintoma visível.
- **CAUSA-RAIZ:** o motivo fundamental que originou o problema, identificado pela análise de causa-raiz.

Exemplo — Do erro à causa-raiz

Um desenvolvedor, com pressa (1), digita ">" onde deveria ser ">=" — isso é o ERRO humano.

O código fica com a comparação errada — isso é o DEFEITO.

Quando um usuário insere exatamente o valor-limite, o sistema rejeita indevidamente o cadastro — isso é a FALHA.

Investigando, descobre-se que a equipe não tinha treinamento sobre análise de valor-limite — essa é a CAUSA-RAIZ. Tratar a causa-raiz (treinar a equipe) evita que defeitos semelhantes se repitam.

 Atenção / Pegadinha

Nem todo defeito vira falha! Alguns defeitos só falham em circunstâncias muito específicas, e alguns NUNCA chegam a ser executados — logo, nunca falham.

E nem toda falha vem de defeito de software: condições ambientais (radiação, campo eletromagnético) podem causar falhas no firmware. A prova testa exatamente essas exceções.

1.3 Os 7 Princípios de Teste

Este é um dos tópicos MAIS cobrados da prova inteira. Quase toda prova tem ao menos uma questão pedindo para identificar um princípio ou reconhecer qual deles se aplica a uma situação. Você precisa saber os sete de cor, com o nome e a ideia.

1 — O teste mostra a presença de defeitos, não a ausência

O teste pode provar que há defeitos, mas nunca pode provar que NÃO há nenhum. Mesmo sem encontrar defeito, o software não está provado correto.

 Analogia — O detetive

Encontrar pegadas prova que alguém esteve no local. Mas NÃO encontrar pegadas não prova que ninguém esteve — talvez tenham apagado os rastros. Testar é procurar pegadas: achar prova a presença; não achar não prova a ausência.

2 — Testes exaustivos são impossíveis

Testar absolutamente todas as combinações de entradas e condições é inviável, exceto em casos triviais. Por isso usamos técnicas de teste, priorização e teste baseado em risco para focar o esforço.

3 — Testes antecipados economizam tempo e dinheiro (shift-left)

Quanto mais cedo um defeito é encontrado, mais barato é corrigi-lo. Por isso o teste — estático e dinâmico — deve começar o mais cedo possível.

 Analogia — O erro na planta

Corrigir um erro na planta da casa (no papel) custa uma borracha. Corrigir o mesmo erro depois que a parede foi levantada custa marreta, pedreiro e material. Antecipar é sempre mais barato.

4 — Os defeitos se agrupam (clustering)

Um pequeno número de módulos costuma concentrar a maioria dos defeitos. É uma aplicação do Princípio de Pareto (regra 80/20). Onde já apareceu muito defeito, tende a aparecer mais.

5 — Os testes se degradam (paradoxo do pesticida)

Se você repetir os mesmos testes muitas vezes, eles param de encontrar defeitos novos — assim como uma praga fica resistente ao mesmo pesticida. Solução: revisar e criar testes novos. (Exceção: regressão automatizada repetida ainda é útil.)

Analogia — O pesticida

Use o mesmo veneno toda safra e os insetos ficam imunes. Para continuar matando pragas, você precisa variar o produto. Testes são iguais: os mesmos casos param de pegar defeitos novos.

6 — Os testes dependem do contexto

Não existe uma abordagem única que sirva para tudo. Testar um app de jogos é diferente de testar um sistema de aviação. O contexto define como testar.

7 — Falácia da ausência de defeitos

É um engano achar que, só porque você corrigiu todos os defeitos, o sistema será um sucesso. Um sistema pode estar tecnicamente sem defeitos e ainda assim ser inútil — porque não atende às necessidades reais do usuário. Por isso, além de verificação, é preciso fazer validação.

Cai na prova

Truque para memorizar os 7 princípios — frase-âncora: "Presentes Exaustos Antecipam Agrupados, Degradando Conforme Falácias". Presença de defeitos / Exaustivo impossível / Antecipado / Agrupam / Degradam / Contexto / Falácia.

Cuidado para não confundir o princípio 1 (presença, não ausência) com o 7 (falácia da ausência). O 1 fala que o teste não prova ausência de defeitos. O 7 fala que mesmo SEM defeitos o sistema pode falhar em atender o usuário.

1.4 Atividades de teste, Testware e Papéis

O teste depende do contexto, mas existe um conjunto comum de atividades que formam o processo de teste. Decore os nomes e a função de cada uma — a prova adora a pergunta "qual atividade responde à pergunta X".

1.4.1 As atividades do processo de teste

Embora pareçam uma sequência, na prática são iterativas e paralelas. São elas:

- **Planejamento de teste:** define os objetivos e escolhe a abordagem dentro das restrições.
- **Monitoramento e Controle de teste:** monitorar = comparar o progresso real com o plano; controlar = tomar as ações corretivas necessárias.
- **Análise de teste:** responde "O QUE testar?". Analisa a base de teste, identifica condições de teste e itens testáveis.
- **Modelagem de teste:** responde "COMO testar?". Transforma condições em casos de teste, identifica itens de cobertura e dados de teste.
- **Implementação de teste:** cria/adquire o material necessário (procedimentos, scripts, dados, ambiente) e organiza o cronograma de execução.

- **Execução de teste:** roda os testes, compara resultado real com esperado, registra resultados e analisa anomalias.
- **Conclusão de teste:** ocorre em marcos (release, fim de iteração). Arquivo testware útil, registra lições aprendidas, gera o relatório de conclusão.

Cai na prova

Duas perguntas que caem quase certo: ANÁLISE de teste responde "o que testar?"; MODELAGEM (design) de teste responde "como testar?". Não inverta. E lembre: monitoramento COMPARA com o plano; controle AGE para corrigir.

1.4.2 O processo de teste no contexto

Os testes não existem isolados. Vários fatores contextuais moldam como se testa. O syllabus lista (vale reconhecer): stakeholders, membros da equipe, domínio do negócio, fatores técnicos, restrições do projeto, fatores organizacionais, o SDLC e as ferramentas disponíveis.

1.4.3 Testware — os produtos de trabalho do teste

Testware é tudo que é PRODUZIDO pelas atividades de teste. Cada atividade gera seu testware:

Atividade	Testware que produz (exemplos)
Planejamento	Plano de teste, cronograma, registro de riscos, critérios de entrada/saída
Monitoramento e Controle	Relatórios de progresso, diretrizes de controle, info de riscos
Análise	Condições de teste (priorizadas), relatórios de defeitos da base de teste
Modelagem	Casos de teste, cartas de teste, itens de cobertura, requisitos de dados/ambiente
Implementação	Procedimentos, scripts automatizados, conjuntos de teste, dados, stubs/drivers
Execução	Registros de teste (logs) e relatórios de defeitos
Conclusão	Relatório de conclusão, lições aprendidas, itens de ação, change requests

Atenção / Pegadinha

Stubs e drivers (e também simuladores e virtualizações de serviço) são exemplos de elementos do AMBIENTE de teste, produzidos na IMPLEMENTAÇÃO. Não confunda com casos de teste (que vêm da modelagem).

1.4.4 Rastreabilidade entre a base de teste e o testware

Rastreabilidade é a capacidade de ligar os elementos entre si: requisito → condição de teste → caso de teste → resultado → defeito. Ela é essencial para o monitoramento e controle eficazes.

Benefícios da boa rastreabilidade:

- Avaliar a cobertura (ex.: os requisitos estão cobertos por casos de teste?).
- Determinar o impacto de mudanças.
- Facilitar auditorias e atender governança de TI.
- Tornar relatórios mais compreensíveis e comunicar status aos stakeholders.

1.4.5 Papéis no teste

O syllabus define DOIS papéis principais (não confunda com cargos — uma mesma pessoa pode acumular os dois):

- **Papel de gerenciamento de teste:** responsabilidade geral pelo processo, equipe e liderança. Foca em planejamento, monitoramento/controle e conclusão.
- **Papel de testador:** responsabilidade pelo lado de engenharia (técnico). Foca em análise, modelagem, implementação e execução.

1.5 Habilidades essenciais e boas práticas

1.5.1 Habilidades genéricas do testador

O syllabus destaca habilidades que, embora genéricas, são especialmente relevantes para testadores:

- Conhecimento sobre testes (técnicas) — aumenta a **EFICÁCIA**.
- Meticulosidade, cuidado, curiosidade, atenção a detalhes, ser metódico.
- Boas habilidades de comunicação, ser bom ouvinte, trabalhar em equipe.
- Pensamento analítico, pensamento crítico, criatividade.
- Conhecimento técnico (ferramentas) — aumenta a **EFICIÊNCIA**.
- Conhecimento do domínio — para entender e se comunicar com o negócio.

Atenção / Pegadinha

Pegadinha de vocabulário: **EFICÁCIA** (effectiveness) = fazer a coisa certa, achar mais defeitos (vem do conhecimento sobre testes). **EFICIÊNCIA** (efficiency) = fazer com menos esforço/recursos (vem do conhecimento técnico/ferramentas). A prova troca esses dois de propósito.

O syllabus também lembra que testadores costumam ser "os portadores de más notícias", e que existe a tendência humana de culpar o mensageiro. Por isso, comunicação é crucial, e os defeitos devem ser comunicados de forma **CONSTRUTIVA**. Há ainda o viés de confirmação, que dificulta aceitar informação que contraria nossas crenças.

1.5.2 Abordagem de equipe completa (whole-team approach)

Prática vinda do Extreme Programming (XP). A ideia: qualquer membro da equipe com o conhecimento necessário pode fazer qualquer tarefa, e **TODOS** são responsáveis pela qualidade. A equipe compartilha o mesmo espaço de trabalho (colocalização) para facilitar a comunicação.

 Atenção / Pegadinha

Atenção ao limite: a abordagem de equipe completa NEM SEMPRE é adequada. Em situações críticas de segurança, por exemplo, pode ser necessário um alto nível de independência de teste — o oposto da equipe completa misturada.

1.5.3 Independência dos testes

Um certo grau de independência torna o testador mais eficaz em achar defeitos, por causa das diferenças de viés cognitivo entre quem cria e quem testa. Mas independência NÃO substitui familiaridade (o próprio desenvolvedor acha muitos defeitos no seu código).

Existem níveis de independência (decore a escala, do menor para o maior):

1. Nenhuma: o autor testa o próprio trabalho.
2. Alguma: colegas da mesma equipe testam.
3. Alta: testadores de fora da equipe, mas dentro da organização.
4. Muito alta: testadores de fora da organização.

Benefícios da independência:

- Reconhece tipos diferentes de falhas e defeitos (visão diferente do desenvolvedor).
- Pode verificar, contestar ou refutar suposições feitas pelos stakeholders.

Desvantagens da independência:

- Isolamento da equipe de desenvolvimento; problemas de comunicação; relação adversa.
- Desenvolvedores podem perder o senso de responsabilidade pela qualidade.
- Testadores independentes podem ser vistos como gargalo ou culpados por atrasos.

 Cai na prova

A boa prática é ter VÁRIOS níveis de independência ao mesmo tempo: desenvolvedores fazem teste de componente; equipe de teste faz teste de sistema; representantes de negócio fazem teste de aceite. A prova pode pedir o nível mais alto de independência: é "testadores de FORA da organização".

Capítulo 2 — Teste ao longo do Ciclo de Vida (SDLC)

Peso na prova: 6 questões (15%). Tema com bastante vocabulário ágil/DevOps. Foque em níveis de teste, tipos de teste e na dupla confirmação x regressão.

2.1 Testes no contexto de um SDLC

SDLC (Software Development Life Cycle) é o ciclo de vida de desenvolvimento de software: um modelo que descreve como as fases de desenvolvimento se relacionam. Existem famílias de modelos:

- **Sequenciais:** cascata (waterfall) e modelo em V. Uma fase termina antes da próxima começar.
- **Iterativos:** espiral, prototipagem. Repetem ciclos.
- **Incrementais:** Processo Unificado. Entregam pedaços (incrementos).

E há práticas/métodos mais detalhados (especialmente ágeis): ATDD, BDD, DDD, XP, FDD, Kanban, Lean IT, Scrum, TDD.

2.1.1 Impacto do SDLC nos testes

A escolha do SDLC impacta diretamente: o escopo e cronograma dos testes, o nível de detalhe da documentação, as técnicas e abordagem, a extensão da automação e o papel/responsabilidades do testador.

- **Em modelos sequenciais:** nas fases iniciais o testador participa de revisões e análise/modelagem; o teste dinâmico só vem mais tarde (quando há código executável).
- **Em modelos iterativos/incrementais:** cada iteração entrega algo funcional, então testes estáticos e dinâmicos acontecem em todos os níveis a cada iteração. Exige feedback rápido e MUITA regressão.
- **Em ágil:** documentação leve, ampla automação de regressão, e mais teste manual baseado em experiência.

2.1.2 Boas práticas de teste (válidas para qualquer SDLC)

- Para cada atividade de desenvolvimento, há uma atividade de teste correspondente.
- Cada nível de teste tem objetivos específicos e diferentes (evita redundância).
- A análise/modelagem de teste começa durante a fase de desenvolvimento correspondente (teste antecipado).
- Testadores revisam os produtos de trabalho assim que os rascunhos ficam disponíveis (apoia o shift-left).

2.1.3 Teste como motivador do desenvolvimento (TDD, ATDD, BDD)

Estas três abordagens têm algo em comum: o teste é escrito ANTES do código. Todas implementam o princípio do teste antecipado e seguem o shift-left.

Abordagem	Como funciona
TDD (Test-Driven Development)	Dirige a codificação por casos de teste. Escreve o teste primeiro, depois o código que o satisfaz, depois refatora.
ATDD (Acceptance Test-Driven Dev.)	Deriva testes dos critérios de aceite, como parte do design do sistema. Testes escritos antes da parte do app ser desenvolvida.
BDD (Behavior-Driven Development)	Expressa o comportamento desejado em linguagem natural simples, no formato Dado/Quando/Então (Given/When/Then). Os casos são depois traduzidos em testes executáveis.

Cai na prova

Associe rápido: BDD = Given/When/Then (Dado/Quando/Então). TDD = código guiado por teste de unidade + refatoração. ATDD = parte dos critérios de aceite. Em todas, os testes podem persistir como testes automatizados para garantir qualidade em futuras refatorações.

2.1.4 DevOps e Testes

DevOps une desenvolvimento (incluindo teste) e operações com objetivos comuns. Exige mudança cultural e se apoia em práticas como Integração Contínua (CI) e Entrega Contínua (CD).

Benefícios do DevOps para o teste:

- Feedback rápido sobre a qualidade do código e impacto de mudanças.
- CI promove o shift-left (devs enviam código com testes de componente e análise estática).
- Ambientes de teste mais estáveis via automação (CI/CD).
- Mais visibilidade de características não funcionais.
- Menos teste manual repetitivo; risco de regressão minimizado pela automação.

Riscos/desafios do DevOps:

- Precisa definir e manter o pipeline e as ferramentas de CI/CD.
- Automação de testes consome recursos e é difícil de manter.

Atenção / Pegadinha

Mesmo com muito teste automatizado, o DevOps NÃO elimina o teste manual — especialmente o teste sob a perspectiva do usuário ainda é necessário. Se a questão disser que DevOps acaba com o teste manual, está errada.

2.1.5 Abordagem Shift-Left

Shift-left é, na prática, o princípio do teste antecipado: "empurrar" o teste para a ESQUERDA na linha do tempo (mais cedo). Importante: shift-left NÃO significa abandonar os testes posteriores.

Boas práticas que ilustram o shift-left:

- Revisar a especificação sob a ótica de testes.
- Escrever casos de teste antes do código.
- Usar CI/CD com feedback rápido e testes de componente automatizados.
- Fazer análise estática antes do teste dinâmico.
- Iniciar testes não funcionais já no nível de componente, quando possível.



Analogia — Pagar a conta antes

Shift-left é como revisar o orçamento de uma reforma ANTES de comprar o material, em vez de descobrir o erro só na hora de pagar. Custa um pouco de esforço extra no começo, mas economiza muito no fim.

2.1.6 Retrospectivas e melhoria de processos

Retrospectivas (reuniões pós-projeto/iteração) reúnem não só testadores, mas devs, arquitetos, PO, analistas. Discutem três perguntas:

1. O que foi bem e deve ser mantido?
2. O que não foi bem e pode melhorar?
3. Como incorporar as melhorias no futuro?

Benefícios típicos: aumento de eficácia/eficiência do teste, melhor qualidade do testware, vínculo e aprendizado da equipe, melhor qualidade da base de teste, melhor cooperação entre dev e teste.

2.2 Níveis de Teste e Tipos de Teste



Atenção / Pegadinha

Esta é a confusão nº 1 do Capítulo 2: NÍVEL de teste ≠ TIPO de teste.

NÍVEL = QUANDO/ONDE no desenvolvimento (componente, integração, sistema, aceite). Está ligado ao estágio.

TIPO = O QUE você verifica (funcional, não funcional, caixa-preta, caixa-branca). Está ligado à característica de qualidade.

Um TIPO pode ser aplicado em qualquer NÍVEL.

2.2.1 Os 5 Níveis de Teste

O syllabus 4.0 define CINCO níveis (na v3 eram quatro — integração foi dividida em duas). Decore na ordem:

Nível	Foco	Quem costuma fazer
Teste de Componente (unidade)	Testar componentes isoladamente. Precisa de frameworks de teste de unidade.	Desenvolvedores
Teste de Integração de Componentes	Interfaces e interações ENTRE componentes. Estratégias: bottom-up, top-down, big-bang.	Devs / equipe técnica
Teste de Sistema	Comportamento geral do sistema inteiro. Funcional ponta-a-ponta + não funcional.	Equipe de teste independente
Teste de Integração de Sistema	Interfaces com OUTROS sistemas e serviços externos.	Equipe de teste
Teste de Aceite	Validação e demonstração de prontidão para implantação (atende ao negócio).	Usuários previstos

As principais formas de Teste de Aceite (decore esta lista, cai):

- UAT — Teste de Aceite do Usuário.
- Teste de Aceite Operacional (ex.: backup/restore, segurança operacional).
- Teste de Aceite Contratual e Normativo (cumpre contrato/regulação).
- Teste Alfa (feito no ambiente do desenvolvedor, por usuários/equipe externa à de desenvolvimento).
- Teste Beta (feito no ambiente do próprio usuário, no campo).

Os níveis são diferenciados por atributos (para evitar sobreposição): objeto de teste, objetivos, base de teste, defeitos/falhas, e abordagem/responsabilidades.

Cai na prova

Alfa x Beta é pegadinha clássica: ALFA acontece no ambiente do DESENVOLVEDOR ("em casa"); BETA acontece no ambiente do CLIENTE/usuário ("no campo"). Lembre: Alfa = Antes, na fábrica. Beta = na casa de quem comprou.

Outra: no modelo sequencial, os critérios de SAÍDA de um nível costumam ser os critérios de ENTRADA do próximo. Em modelos iterativos isso pode não valer, e os níveis podem se sobrepor no tempo.

2.2.2 Os 4 Tipos de Teste

O syllabus aborda quatro tipos:

- **Teste Funcional:** avalia O QUE o sistema faz (as funções). Verifica integridade, correção e adequação funcional.
- **Teste Não Funcional:** avalia QUÃO BEM o sistema se comporta. Verifica as características não funcionais de qualidade.

- **Teste Caixa-Preta:** baseado em especificações (visão externa). Verifica comportamento contra a especificação.
- **Teste Caixa-Branca:** baseado na estrutura interna (código, arquitetura). Objetivo: cobrir a estrutura interna.

As 8 características não funcionais de qualidade (norma ISO/IEC 25010) — vale reconhecer:

- Eficiência de performance, Compatibilidade, Usabilidade, Confiabilidade, Segurança, Capacidade de manutenção (manutenibilidade), Portabilidade.

Atenção / Pegadinha

A prova pode perguntar de qual NORMA vêm as características de qualidade: é a ISO/IEC 25010. E lembre que os quatro tipos de teste podem ser aplicados em TODOS os níveis de teste — o foco é que muda em cada nível.

2.2.3 Teste de Confirmação x Teste de Regressão

Já vimos rapidamente no Cap. 1. Aqui aprofundamos, porque é altamente cobrado:

	Teste de Confirmação (reteste)	Teste de Regressão
Objetivo	Confirmar que o defeito ORIGINAL foi corrigido.	Confirmar que a mudança NÃO causou efeitos adversos em outras partes.
Pergunta	O bug que eu reporteii sumiu?	A correção quebrou outra coisa?
Alvo	O caso que falhava antes.	Outras áreas, outros componentes, até outros sistemas e o ambiente.

Analogia — O conserto do encanamento

Você chama o encanador para consertar uma torneira que pingava. Abrir a torneira e ver que parou de pingar é o TESTE DE CONFIRMAÇÃO (o problema específico foi resolvido?). Ir ao resto da casa checar se, ao mexer no cano, ele não causou vazamento na parede do quarto ao lado é o TESTE DE REGRESSÃO (a correção estragou outra coisa?).

Sobre regressão, decore também:

- Conjuntos de regressão crescem a cada iteração → são fortes candidatos à AUTOMAÇÃO.
- Recomenda-se fazer análise de impacto antes, para otimizar o escopo da regressão (mostra quais partes podem ser afetadas).
- Confirmação e/ou regressão são necessários em TODOS os níveis sempre que defeitos forem corrigidos ou mudanças forem feitas.

2.3 Teste de Manutenção

Manutenção é o teste das mudanças feitas em um sistema que JÁ está em produção. Antes de mudar, pode-se fazer análise de impacto para decidir se vale a pena mudar.

O escopo do teste de manutenção depende de três coisas:

1. O grau de risco da mudança.
2. O tamanho do sistema existente.
3. O tamanho da mudança.

Os gatilhos (fatores) da manutenção classificam-se em:

- **Modificações:** melhorias planejadas (por release), correções, ou hot fixes (correções não planejadas).
- **Migração/atualização de ambiente:** mudar de plataforma; exige testes do novo ambiente e, às vezes, testes de conversão de dados.
- **Aposentadoria (retirement):** fim de vida do sistema. Pode exigir testes de arquivamento de dados e de restauração/recuperação.

Cai na prova

Pergunta comum: "O que define o escopo do teste de manutenção?" → grau de risco da mudança + tamanho do sistema + tamanho da mudança. E "hot fix" é classificado como uma MODIFICAÇÃO (correção não planejada).

Capítulo 3 — Teste Estático

Peso na prova: 4 questões (10%). Capítulo curto. O ouro aqui é: tipos de revisão, papéis na revisão e a diferença estático x dinâmico.

3.1 Noções básicas de Teste Estático

No teste estático, o software NÃO é executado. Você examina o artefato — código, especificação, arquitetura — de duas formas:

- **Exame manual:** as revisões (feitas por pessoas).
- **Com ferramenta:** a análise estática (feita por software, sobre algo com estrutura formal, como código).

O teste estático pode servir tanto para verificação quanto para validação. E é onde os testadores entram cedo: revisando histórias de usuário, participando do mapeamento de exemplos e do refinamento de backlog, fazendo as perguntas certas para melhorar as histórias.

A análise estática, frequentemente embutida na CI, detecta defeitos no código e também avalia manutenibilidade e segurança. Corretores ortográficos e ferramentas de legibilidade também são exemplos de análise estática.

3.1.1 O que pode ser examinado por teste estático

Quase qualquer produto de trabalho que possa ser LIDO e COMPREENDIDO pode ser revisado: requisitos, código-fonte, planos de teste, casos de teste, itens de backlog, cartas de teste, documentação de projeto, contratos, modelos.

Atenção / Pegadinha

Distinção importante: para a ANÁLISE ESTÁTICA (com ferramenta), o produto precisa de uma ESTRUTURA FORMAL contra a qual ser verificado (código, modelos, texto com sintaxe formal). Já a REVISÃO (manual) aceita qualquer coisa legível.

O que NÃO serve para teste estático: artefatos que humanos não conseguem interpretar e que ferramentas não conseguem analisar — ex.: código executável de terceiros (por motivos legais).

3.1.2 Valor do teste estático

- Detecta defeitos cedo (cumprir o teste antecipado / shift-left).
- Encontra defeitos que o teste dinâmico NÃO acha (ex.: código inacessível, padrões de projeto mal implementados, defeitos em artefatos não executáveis).
- Cria entendimento compartilhado e melhora a comunicação entre stakeholders.
- Reduz o custo geral do projeto (corrigir cedo é mais barato).

3.1.3 Diferenças entre estático e dinâmico

Característica	Teste Estático	Teste Dinâmico
Executa o software?	Não	Sim
Como acha defeitos	Encontra o DEFEITO diretamente	Causa uma FALHA; o defeito é deduzido depois
Tipos de defeito	Pega defeitos em caminhos raramente executados; em artefatos não executáveis	Pega defeitos que dependem da execução
Qualidade que mede	Não dependente de execução (ex.: manutenibilidade)	Dependente de execução (ex.: performance)

Defeitos típicos que o teste estático acha mais fácil/barato:

- Defeitos em requisitos (ambiguidades, contradições, omissões, duplicações).
- Defeitos de projeto (banco de dados ineficiente, modularização ruim).
- Defeitos de codificação (variáveis não declaradas, código inacessível, complexidade excessiva).
- Desvios de padrões (não seguir convenções de nomenclatura).
- Especificações de interface incorretas (parâmetros incompatíveis).
- Vulnerabilidades de segurança (ex.: buffer overflow).
- Lacunas na cobertura da base de teste.

Cai na prova

Frase de ouro: "O teste ESTÁTICO encontra o DEFEITO diretamente; o teste DINÂMICO causa uma FALHA, e o defeito é descoberto por análise posterior." Essa frase, sozinha, responde várias questões.

3.2 Processo de feedback e revisão

3.2.1 Benefícios do feedback antecipado e frequente

Feedback cedo e frequente dos stakeholders evita mal-entendidos sobre requisitos, garante que mudanças sejam compreendidas cedo, e foca a equipe no que agrega mais valor. Sem isso, o produto pode não atender à visão dos stakeholders — gerando retrabalho caro, atrasos e até o fracasso do projeto.

3.2.2 As atividades do processo de revisão

A norma ISO/IEC 20246 define um processo genérico de revisão. Quanto mais formal, mais tarefas. As atividades são:

1. Planejamento — define escopo, objetivo, o que revisar, características a avaliar, critérios de saída, esforço e prazos.

2. Início da revisão (kickoff) — garante que todos estão preparados, têm acesso ao artefato e entendem seus papéis.
3. Revisão individual — cada revisor avalia sozinho e registra anomalias, recomendações e perguntas (usando técnicas como checklist ou cenário).
4. Comunicação e análise de problemas — discute-se cada anomalia (nem toda anomalia é defeito!); decide-se status, propriedade e ações. Geralmente numa reunião de revisão.
5. Correção e relatório — cria-se relatório de defeitos para cada defeito; quando os critérios de saída são atingidos, o artefato é aceito.

Atenção / Pegadinha

Conceito-chave: uma ANOMALIA não é necessariamente um DEFEITO. Anomalias precisam ser analisadas e discutidas antes de virarem (ou não) defeitos. Isso vale tanto na revisão quanto no gerenciamento de defeitos (Cap. 5).

3.2.3 Papéis e responsabilidades nas revisões

Este é o tópico que MAIS cai do Capítulo 3. Decore cada papel e sua responsabilidade:

Papel	Responsabilidade
Gerente (Manager)	Decide o que será revisado e fornece recursos (equipe, tempo).
Autor (Author)	Cria e corrige o produto de trabalho em análise.
Moderador / Facilitador	Garante o andamento eficaz das reuniões; mediação, gestão de tempo, ambiente seguro.
Relator / Registrador (Scribe)	Reúne as anomalias dos revisores e registra as informações da revisão.
Revisor (Reviewer)	Realiza as revisões. Pode ser especialista no assunto ou outra parte interessada.
Líder da revisão (Review leader)	Responsabilidade geral: decide quem participa, quando e onde a revisão acontece.

Cai na prova

Pegadinha de inspeção: na INSPEÇÃO (revisão mais formal), o AUTOR não pode atuar como líder nem como relator da revisão. Essa restrição cai com frequência.

Não confunda Moderador (toca a reunião) com Líder da revisão (organiza a revisão como um todo) nem com Relator (anota).

3.2.4 Tipos de revisão

Vão da mais informal à mais formal. O nível de formalidade depende do SDLC, da maturidade do processo, da criticidade do artefato e de exigências legais/auditoria. Os quatro principais:

Tipo	Formalidade	Liderado por	Objetivo principal
Revisão Informal	Nenhum processo definido; sem resultado documentado obrigatório	—	Detectar anomalias
Walkthrough (Passo a passo)	Variável	AUTOR	Avaliar qualidade, gerar consenso/ideias, instruir; revisão individual prévia é opcional
Revisão Técnica	Formal	Moderador (revisores tecnicamente qualificados)	Obter consenso e decidir sobre questão técnica; também detectar anomalias
Inspeção	A MAIS formal	Moderador (segue o processo completo)	Encontrar o número MÁXIMO de anomalias; coleta métricas para melhorar o SDLC

Cai na prova

Memorize quem lidera: WALKTHROUGH é liderado pelo AUTOR. INSPEÇÃO é a mais formal e segue o processo genérico completo. INFORMAL não tem processo definido. O objetivo da INSPEÇÃO é achar o MÁXIMO de anomalias.

3.2.5 Fatores de sucesso para revisões

Reconheça estes fatores (cai em questão de "qual NÃO é um fator de sucesso"):

- Definir objetivos claros e critérios de saída mensuráveis.
- Escolher o tipo de revisão apropriado ao objetivo e contexto.
- Fazer revisões em partes pequenas (manter a concentração).
- Dar feedback aos stakeholders e autores.
- Dar tempo suficiente de preparação.
- Apoio da gerência ao processo de revisão.
- Tornar a revisão parte da cultura (promover aprendizado).
- Treinar os participantes em seus papéis.
- Facilitar bem as reuniões.

Atenção / Pegadinha

Fator de sucesso explícito no syllabus: "a avaliação dos PARTICIPANTES nunca deve ser um objetivo". Ou seja, revisão não serve para avaliar pessoas. Questão clássica: se uma alternativa disser que o objetivo é avaliar o desempenho do autor, está ERRADA.

Capítulo 4 — Análise e Modelagem de Teste

Peso na prova: 11 questões (27,5%) — O MAIOR PESO da prova. É o único capítulo com questões K3 (aplicar), que pedem cálculo. Aqui você não pode chutar: precisa SABER derivar casos de teste. Treine com exercícios.

4.1 Visão geral das técnicas de teste

As técnicas de teste ajudam o testador a fazer duas coisas de forma sistemática: a análise (o que testar) e a modelagem/design (como testar), produzindo um conjunto pequeno mas suficiente de casos de teste. Elas se dividem em três famílias:

Família	Também chamada de	Baseada em quê?	Quando os casos podem ser criados
Caixa-Preta	Baseada em especificações	Comportamento externo, SEM olhar a estrutura interna	A qualquer momento (independem da implementação)
Caixa-Branca	Baseada na estrutura	Estrutura/processamento interno (código)	Só APÓS o design/implementação
Baseada na Experiência	—	Conhecimento e experiência do testador	Depende da habilidade do testador

Cai na prova

Caixa-preta: se a IMPLEMENTAÇÃO mudar mas o comportamento exigido continuar o mesmo, os casos ainda valem. Caixa-branca: como dependem do código, só dá para criar depois que o código existe. As técnicas baseadas em experiência são COMPLEMENTARES (acham defeitos que as outras não acham).

4.2 Técnicas de Teste Caixa-Preta (K3 — você vai CALCULAR)

São quatro técnicas. Para cada uma, o que cai é: como derivar os casos e como calcular a cobertura. Vamos com calma e exemplos numéricos.

4.2.1 Particionamento de Equivalência (EP)

A ideia: dividir os dados em partições (grupos) em que todos os valores são tratados da MESMA forma pelo sistema. A teoria diz que, se um valor de uma partição acha um defeito, qualquer outro valor da mesma partição também acharia. Logo, basta UM teste por partição.

As partições:

- Não devem se sobrepor e não podem ser vazias.
- **Partição VÁLIDA** = valores que o sistema deve aceitar/processar.

- **Partição INVÁLIDA** = valores que o sistema deve rejeitar/ignorar.

Cobertura: itens de cobertura = as partições. Para 100% de cobertura, cada partição (válidas E inválidas) deve ser executada ao menos uma vez. Cobertura = partições executadas ÷ total de partições × 100%.

Exemplo — EP na prática — idade de um seguro

Regra: o seguro aceita idades de 18 a 65 anos. Quais as partições?

Partição inválida 1: idade < 18 (ex.: 10).

Partição válida: $18 \leq \text{idade} \leq 65$ (ex.: 30).

Partição inválida 2: idade > 65 (ex.: 80).

São 3 partições → bastam 3 casos de teste (um valor de cada) para 100% de cobertura. Não importa se você testa 30 ou 45 na válida: ambos estão na mesma partição.

Quando há vários parâmetros de entrada (vários conjuntos de partições), o critério mais simples é o Each Choice Coverage (ECC): cada partição de cada conjunto deve ser executada ao menos uma vez. O ECC NÃO testa combinações de partições.

Cai na prova

Erro comum: esquecer de contar as partições INVÁLIDAS. 100% de cobertura EP exige cobrir as válidas E as inválidas. Se o problema dá um intervalo "de X a Y", quase sempre há 3 partições: abaixo, dentro, acima.

4.2.2 Análise de Valor de Limite (BVA)

BVA complementa o EP focando nos LIMITES (bordas) das partições — porque é exatamente nas bordas que os programadores mais erram (trocar > por >=, etc.). Importante: BVA só funciona em partições ORDENADAS (que têm noção de mínimo e máximo).

O syllabus cobra DUAS versões. Saber diferenciá-las é essencial:

	BVA de 2 valores	BVA de 3 valores
Itens por limite	2: o valor-limite + o vizinho da partição adjacente	3: o valor-limite + os dois vizinhos (de cada lado)
Rigor	Menos rigoroso	Mais rigoroso (acha defeitos que o de 2 não acha)

Exemplo — BVA — campo que aceita de 1 a 10

A partição válida é [1..10]. Os limites são 1 e 10.

BVA de 2 valores → para cada limite, o valor e o vizinho de fora:

Limite 1 → testar 0 e 1. Limite 10 → testar 10 e 11.

Valores: 0, 1, 10, 11 (4 valores de teste).

BVA de 3 valores → para cada limite, o valor e os DOIS vizinhos:

Limite 1 → testar 0, 1, 2. Limite 10 → testar 9, 10, 11.

Valores: 0, 1, 2, 9, 10, 11 (6 valores de teste).

Por que o de 3 valores é melhor? O syllabus dá um exemplo memorável: se a regra "se $(x \leq 10)$ " for implementada por engano como "se $(x = 10)$ ", os valores do BVA de 2 valores (10 e 11) NÃO detectam o defeito. Mas o valor 9 — que só existe no BVA de 3 valores — detecta.

Cai na prova

Pergunta típica K3: "Quantos valores de teste para 100% de cobertura BVA de 3 valores?" Conte: por limite, 3 itens; mas vizinhos podem se sobrepor entre limites próximos. Sempre liste os valores e remova repetições antes de contar. E lembre: BVA só se aplica a partições ORDENADAS.

4.2.3 Teste de Tabela de Decisão

Serve para testar regras de negócio: situações em que diferentes COMBINAÇÕES de condições levam a diferentes resultados (ações). É a melhor técnica para registrar lógica complexa.

Como ler a tabela:

- Linhas de cima = condições. Linhas de baixo = ações.
- Cada coluna = uma regra de decisão (uma combinação única de condições + suas ações).

A notação que você precisa reconhecer:

- **V (verdadeiro)** — condição satisfeita. **F (falso)** — não satisfeita.
- **“—”** — valor da condição é irrelevante (don't care) para aquela regra.
- **N/A** — condição inviável para aquela regra.
- **X (em ações)** — a ação deve ocorrer. Em branco — a ação não ocorre.

Exemplo — Tabela de decisão — desconto de loja

Regra de negócio: cliente VIP ganha 10% de desconto; compra acima de R\$500 ganha 5%; se for VIP E acima de 500, ganha os dois.

Condições: É VIP? / Compra > 500?

Com 2 condições booleanas → $2^2 = 4$ combinações (4 colunas/regras):

Regra 1: VIP=V, >500=V → aplica 10% e 5%.

Regra 2: VIP=V, >500=F → aplica 10%.

Regra 3: VIP=F, >500=V → aplica 5%.

Regra 4: VIP=F, >500=F → nenhum desconto.

Cada regra vira ao menos 1 caso de teste.

Cobertura: itens de cobertura = colunas com combinações VIÁVEIS de condições. 100% = executar todas essas colunas. A tabela pode ser simplificada (removendo combinações inviáveis) ou minimizada (juntando colunas onde a condição não importa).

Cai na prova

Cálculo-chave: com N condições booleanas, a tabela COMPLETA tem 2^N colunas. 2 condições = 4; 3 condições = 8; 4 condições = 16. O número de regras cresce EXPONENCIALMENTE — por isso, se há muitas condições, usa-se tabela minimizada ou abordagem baseada em risco. Ponto forte da técnica: identifica sistematicamente combinações que poderiam ser esquecidas, e revela lacunas/contradições nos requisitos.

4.2.4 Teste de Transição de Estado

Modela o comportamento do sistema em termos de ESTADOS e as TRANSIÇÕES entre eles. Uma transição é disparada por um EVENTO, pode ter uma CONDIÇÃO DE PROTEÇÃO (guard) e pode gerar uma AÇÃO. A sintaxe do rótulo: "evento [condição de proteção] / ação".

Dois modelos equivalentes:

- **Diagrama de transição de estado:** o desenho com bolinhas (estados) e setas (transições).
- **Tabela de estados:** linhas = estados, colunas = eventos. As células mostram a transição. Diferença importante: a TABELA mostra explicitamente as transições INVÁLIDAS (células vazias).

Analogia — A catraca do metrô

A catraca tem dois estados: Travada e Destravada. Evento "inserir bilhete" na catraca Travada → transita para Destravada. Evento "empurrar" na Destravada → volta para Travada. Empurrar quando está Travada não faz nada (transição inválida). É exatamente assim que se modela transição de estado.

O syllabus cobra TRÊS critérios de cobertura — você precisa diferenciá-los e saber qual é mais forte:

Critério	Itens de cobertura	Força
Cobertura de TODOS OS ESTADOS	Todos os estados são visitados	Mais FRACO (pode ser atingido sem percorrer todas as transições)
Cobertura de TRANSIÇÕES VÁLIDAS (0-switch)	Cada transição válida única	Intermediário; é o critério MAIS usado

Critério	Itens de cobertura	Força
Cobertura de TODAS AS TRANSIÇÕES	Todas as transições, incluindo as INVÁLIDAS (tentando executá-las)	Mais FORTE; mínimo para software crítico

Cai na prova

Hierarquia que cai: alcançar a cobertura de TODAS AS TRANSIÇÕES garante automaticamente a cobertura de todos os estados E de transições válidas. "Todos os estados" é o critério MAIS FRACO. "Transições válidas" (também chamada 0-switch) é a mais usada. Para software de missão/segurança crítica, o mínimo é "todas as transições".

Detalhe: ao testar transições inválidas, testa-se UMA por caso de teste para evitar o mascaramento de falhas (um defeito esconder outro).

4.3 Técnicas de Teste Caixa-Branca (K2 — só EXPLICAR)

Boa notícia: caixa-branca no Foundation é nível K2 (explicar), não K3. Você não precisa desenhar grafos de fluxo, só entender os conceitos. O syllabus foca em duas técnicas baseadas em código.

4.3.1 Teste de Instrução e Cobertura de Instrução

Itens de cobertura = as instruções (statements) executáveis. Objetivo: exercitar as instruções até atingir uma cobertura aceitável. Cobertura = instruções executadas ÷ total de instruções × 100%.

Com 100% de cobertura de instrução, toda instrução executável rodou ao menos uma vez. MAS isso não garante que toda a lógica de decisão foi testada (ex.: pode não executar todos os ramos de um if), nem pega defeitos que dependem de dados (ex.: divisão por zero que só falha com denominador zero).

4.3.2 Teste de Ramificação e Cobertura de Ramificação

Uma ramificação (branch) é uma transferência de controle entre dois nós do grafo de fluxo. Pode ser incondicional (código em linha reta) ou condicional (resultado de uma decisão: o verdadeiro/falso de um if, os casos de um switch, sair/continuar um loop).

Itens de cobertura = as ramificações. 100% de cobertura de ramificação = todas as ramificações (condicionais e incondicionais) foram exercitadas.

Cai na prova

A relação MAIS cobrada do tópico: a cobertura de RAMIFICAÇÃO SUBSUME (substitui/inclui) a cobertura de INSTRUÇÃO. Ou seja: 100% de ramificação ⇒ 100% de instrução. Mas o contrário NÃO é verdade: 100% de instrução NÃO garante 100% de ramificação.

Curiosidade do syllabus 4.0: trocaram "cobertura de decisão" por "cobertura de ramificação" justamente porque "100% decisão ⇒ 100% instrução" não vale para programas sem decisões. Com ramificação, a regra sempre vale.

4.3.3 O valor do teste caixa-branca

- **Ponto forte:** considera TODA a implementação; acha defeitos mesmo quando a especificação é vaga, desatualizada ou incompleta.
- **Ponto fraco:** se o software NÃO implementou um requisito, a caixa-branca pode não detectar essa ausência (não há código para cobrir).
- Pode ser usada em teste estático (ex.: dry runs de código).
- Fornece medida OBJETIVA de cobertura — coisa que só caixa-preta não dá.

4.4 Técnicas Baseadas na Experiência (K2)

Usam o conhecimento e a intuição do testador. A eficácia depende muito da habilidade de quem testa. São complementares às caixa-preta e caixa-branca.

4.4.1 Suposição de Erro (Error Guessing)

O testador PREVÊ onde podem estar erros, defeitos e falhas, com base em: como o app se comportou no passado, que tipos de erro os devs costumam cometer, e que falhas ocorreram em apps semelhantes.

Áreas comuns onde "adivinhar": entrada (entrada correta não aceita, parâmetros errados/ausentes), saída (formato/resultado errado), lógica (casos ausentes, operador errado), cálculo (operando/cálculo errado), interfaces (parâmetros incompatíveis) e dados (inicialização incorreta, tipo errado).

Ataques de falha (fault attacks) são a forma METÓDICA de implementar a suposição de erro: o testador cria/adquire uma lista de possíveis erros/defeitos/falhas e modela testes para expô-los.

4.4.2 Teste Exploratório

Aqui os testes são modelados, executados e avaliados AO MESMO TEMPO, enquanto o testador APRENDE sobre o objeto de teste. Não há um roteiro pronto; o aprendizado guia os próximos testes.

Frequentemente é estruturado em teste baseado em sessões (session-based): um período de tempo fixo, guiado por uma carta de teste (test charter) com os objetivos. Depois há uma reunião de informe (debriefing). O testador pode usar fichas de sessão para documentar.

É útil quando: há poucas/inadequadas especificações, ou há forte pressão de tempo. Também complementa técnicas mais formais. Funciona melhor com testadores experientes, com conhecimento do domínio e boas habilidades essenciais (analítico, curioso, criativo).



Analogia — Explorar uma cidade nova

Teste com roteiro é seguir um tour com itinerário fixo. Teste exploratório é andar pela cidade sem mapa fechado: você vê uma rua interessante, entra, descobre uma praça, e isso te leva a explorar o próximo beco. Você aprende e decide para onde ir ao mesmo tempo.

4.4.3 Teste Baseado em Lista de Verificação (Checklist)

O testador cobre as condições de teste de uma checklist. As listas vêm da experiência, do conhecimento do que importa ao usuário, ou de como o software costuma falhar. Os itens geralmente são perguntas, verificáveis individualmente.

Cuidados que o syllabus aponta (e que caem):

- Itens não devem ser verificáveis automaticamente, nem ser critérios de entrada/saída, nem ser genéricos demais.
- Listas se degradam com o tempo (devs aprendem a evitar os mesmos erros) → atualizar regularmente com base na análise de defeitos.
- Evitar que a checklist fique longa demais.
- Em listas de alto nível, há mais variabilidade → cobertura potencialmente maior, mas menos repetibilidade.

4.5 Abordagens Baseadas na Colaboração

Enquanto as técnicas anteriores focam em DETECTAR defeitos, as abordagens colaborativas focam também em PREVENIR defeitos, por meio de comunicação e colaboração.

4.5.1 Escrita colaborativa de histórias de usuário

Uma história de usuário representa uma funcionalidade valiosa para o usuário. Tem três aspectos críticos, os "3 Cs":

- **Cartão (Card):** o meio onde a história é registrada.
- **Conversação (Conversation):** a explicação de como o software será usado (verbal ou documentada).
- **Confirmação (Confirmation):** os critérios de aceite.

Formato mais comum: "Como [ator/persona], quero [meta], para que eu possa [valor de negócio]", seguido dos critérios de aceite.

A colaboração reúne TRÊS perspectivas: negócio, desenvolvimento e teste. E boas histórias seguem o acrônimo INVEST:

Letra	Significado
I — Independent	Independente de outras histórias
N — Negotiable	Negociável
V — Valuable	Valiosa para o usuário
E — Estimable	Estimável (dá para estimar o esforço)
S — Small	Pequena
T — Testable	Testável

**Cai na prova**

Os 3 Cs (Card, Conversation, Confirmation) e o INVEST caem com frequência. Macete: se um stakeholder NÃO sabe como testar uma história, isso é sinal de que ela não está clara o suficiente, não reflete algo valioso, ou ele precisa de ajuda para testar — o 'T' (Testable) do INVEST.

4.5.2 Critérios de Aceite

São as condições que a implementação da história precisa cumprir para ser aceita pelos stakeholders. Vistos como as condições de teste que os testes devem executar. Normalmente resultam da Conversação (o segundo C).

Para que servem:

- Definir o escopo da história.
- Chegar a consenso entre stakeholders.
- Descrever cenários positivos e negativos.
- Servir de base para o teste de aceite (ATDD).
- Permitir planejamento e estimativa precisos.

Dois formatos mais comuns para escrever critérios de aceite:

- **Orientado a cenários:** formato Given/When/Then (Dado/Quando/Então), usado no BDD.
- **Orientado por regras:** lista de pontos de verificação ou tabela de mapeamento entrada→saída.

4.5.3 Desenvolvimento Orientado por Teste de Aceite (ATDD) — K3

ATDD prioriza o teste: os casos de teste são criados ANTES de implementar a história. São criados de forma colaborativa por gente com perspectivas diferentes (cliente, dev, testador). Podem ser manuais ou automatizados.

O processo do ATDD:

1. Workshop de especificação: a história e seus critérios de aceite são analisados, discutidos e escritos. Lacunas/ambiguidades são resolvidas aqui.
2. Criação dos casos de teste: baseados nos critérios de aceite (podem ser vistos como exemplos de como o software funciona).
3. Primeiro os casos POSITIVOS (comportamento correto, sem erros), depois os NEGATIVOS, e por fim os não funcionais.

Regras importantes do ATDD (caem em K3):

- Os casos devem ser compreensíveis pelos stakeholders (frases em linguagem natural com pré-condições, entradas e pós-condições).
- Os casos devem cobrir TODAS as características da história — e NÃO ir além dela.
- Dois casos de teste NÃO devem descrever as mesmas características da história.

- Quando capturados em formato compatível com automação, viram "requisitos executáveis".

 Cai na prova

Em ATDD: "exemplos" e "testes" são a mesma coisa (termos intercambiáveis). Ordem dos casos: positivos → negativos → não funcionais. Os casos não podem extrapolar a história nem se repetir entre si.

Capítulo 5 — Gerenciamento das Atividades de Teste

Peso na prova: 9 questões (22,5%) — o segundo maior peso. Tem K3 em estimativa, priorização e relatório de defeitos. Risco e planejamento dominam as questões.

5.1 Planejamento de Teste

5.1.1 Objetivo e conteúdo de um plano de teste

Um plano de teste descreve os objetivos, recursos e processos de um projeto de teste. Ele:

- Documenta os meios e o cronograma para atingir os objetivos.
- Ajuda a garantir que as atividades atenderão aos critérios estabelecidos.
- Serve de comunicação com a equipe e stakeholders.
- Demonstra que os testes seguirão a política e a estratégia (ou explica por que vão se desviar).

O processo de PREPARAR o plano já é valioso: força o testador a pensar em riscos, cronograma, pessoas, ferramentas, custos e esforço. Conteúdo típico: contexto do teste, premissas/restrições, stakeholders, comunicação, registro de riscos, abordagem de teste e orçamento/cronograma.

5.1.2 Contribuição do testador no planejamento de iteração e liberação

Em SDLCs iterativos há dois planejamentos:

- **Planejamento de liberação (release):** olha o lançamento do produto, define/refina o backlog. O testador ajuda a escrever histórias testáveis, participa da análise de risco, estima o esforço de teste e define a abordagem para a versão.
- **Planejamento de iteração:** olha o fim de uma única iteração (backlog da iteração). O testador faz análise de risco detalhada, avalia a testabilidade das histórias, divide em tarefas, estima e refina os aspectos funcionais/não funcionais.

5.1.3 Critérios de Entrada e Critérios de Saída

Conceito muito cobrado, inclusive com os termos ágeis associados:

- **Critérios de ENTRADA:** as pré-condições para INICIAR uma atividade. Sem eles, a atividade fica mais difícil, cara e arriscada. Ex.: ambiente pronto, dados disponíveis, smoke test passou.
- **Critérios de SAÍDA:** o que precisa ser alcançado para DECLARAR uma atividade concluída. Ex.: cobertura atingida, nº de defeitos não resolvidos aceitável, testes planejados executados.

Os termos ágeis equivalentes (decore!):

Termo ágil	Equivale a	O que define
DoR — Definition of Ready (Definição de Pronto)	Critério de ENTRADA	O que uma história precisa cumprir para começar a ser desenvolvida/testada
DoD — Definition of Done (Definição de Feito)	Critério de SAÍDA	As métricas objetivas para um item ser considerado entregável

⚠️ Atenção / Pegadinha

Pegadinha frequente: DoR (Ready) = ENTRADA. DoD (Done) = SAÍDA. É fácil trocar. Macete: 'Ready' (pronto para COMEÇAR) = entrada; 'Done' (feito, ACABOU) = saída.

Outro ponto: o esgotamento de tempo ou orçamento PODE ser um critério de saída válido — desde que os stakeholders aceitem o risco de ir para produção sem mais testes.

5.1.4 Técnicas de Estimativa (K3)

São QUATRO técnicas. Duas são baseadas em métricas e duas em especialistas. A de três pontos é a única com CÁLCULO — e cai.

Técnica	Base	Como funciona
Estimativa baseada em índices (ratios)	Métricas	Usa proporções de projetos anteriores. Ex.: se dev:teste foi 3:2 e dev agora é 600h, teste = 400h.
Extrapolação	Métricas	Mede cedo no projeto atual e extrapola o restante (bom em iterativo). Ex.: média das 3 últimas iterações.
Wideband Delphi	Especialistas	Cada especialista estima isolado; discutem divergências; reestimam até consenso. Planning Poker é uma variante ágil.
Estimativa de três pontos	Especialistas	Usa otimista (o), mais provável (m) e pessimista (p). Calcula média ponderada.

💡 Exemplo — Estimativa de três pontos — a fórmula que cai

Fórmula da estimativa: $E = (o + 4m + p) / 6$

Fórmula do desvio (erro de medição): $SD = (p - o) / 6$

Exemplo do syllabus: $o = 6$, $m = 9$, $p = 18$ (em homens/hora).

$E = (6 + 4 \times 9 + 18) / 6 = (6 + 36 + 18) / 6 = 60 / 6 = 10$.

$SD = (18 - 6) / 6 = 12 / 6 = 2$.

Resultado: 10 ± 2 homens/hora (ou seja, entre 8 e 12).

🎯 Cai na prova

DECORE a fórmula: $E = (o + 4m + p) / 6$. O 'm' (mais provável) tem peso 4. Várias provas trazem uma questão pedindo esse cálculo com números diferentes. Treine substituir os valores rapidamente.

5.1.5 Priorização de casos de teste (K3)

Depois de prontos, os casos são organizados num cronograma de execução. As três estratégias de priorização mais comuns:

- **Baseada em RISCO:** executa primeiro os casos que cobrem os riscos mais importantes (vem da análise de risco).
- **Baseada em COBERTURA:** executa primeiro os casos que atingem maior cobertura. Variante: cobertura adicional (cada caso seguinte é o que adiciona mais cobertura nova).
- **Baseada em REQUISITOS:** executa primeiro os casos ligados aos requisitos mais prioritários (prioridade definida pelos stakeholders).

Atenção / Pegadinha

Limitação importante: a priorização ideal pode não funcionar se houver DEPENDÊNCIAS. Se um caso de alta prioridade depende de um de baixa prioridade, o de baixa tem de rodar primeiro. A ordem de execução também depende da disponibilidade de recursos (ferramentas, ambientes, pessoas).

5.1.6 Pirâmide de Teste

Modelo que mostra que diferentes testes têm diferentes GRANULARIDADES. Ajuda na automação e na alocação de esforço.

- Camada de BAIXO: testes pequenos, isolados, rápidos (ex.: unidade). Verificam pouca funcionalidade → precisa de MUITOS deles.
- Camada de CIMA: testes complexos, de alto nível, ponta-a-ponta. Lentos, verificam muita funcionalidade → precisa de POUCOS deles.

O modelo original (Cohn) tem 3 camadas: testes de unidade, testes de serviço, testes de interface do usuário. (O número e os nomes das camadas podem variar.)

Analogia — A pirâmide de comida

Lembra a pirâmide alimentar: a base larga (testes de unidade, baratos e numerosos) você consome muito; o topo estreito (testes ponta-a-ponta, caros e lentos) você consome pouco. Quanto mais alto, menos granular, mais lento e em menor quantidade.

5.1.7 Quadrantes de Teste

Modelo de Brian Marick (popularizado por Crispin/Gregory) que agrupa níveis, tipos e atividades de teste em quatro quadrantes, cruzando dois eixos: voltado para Negócio x Tecnologia, e Apoia a equipe x Critica/avalia o produto.

Quadrante	Eixos	Contém	Auto/Manual
Q1	Tecnologia + Apoia a equipe	Testes de componente e de integração de componentes	Automatizados (no CI)

Quadrante	Eixos	Contém	Auto/Manual
Q2	Negócio + Apoia a equipe	Testes funcionais, exemplos, testes de história, protótipos de UX, testes de API, simulações	Manuais ou automatizados
Q3	Negócio + Avalia o produto	Testes exploratórios, de usabilidade, de aceite do usuário	Geralmente manuais
Q4	Tecnologia + Avalia o produto	Smoke tests e testes não funcionais (exceto usabilidade)	Geralmente automatizados

Cai na prova

Macete dos quadrantes: Q1 e Q2 APOIAM a equipe (guiam o desenvolvimento); Q3 e Q4 AVALIAM/criticam o produto. Q1 e Q4 são voltados à TECNOLOGIA; Q2 e Q3 ao NEGÓCIO. Exploratório e usabilidade caem no Q3. Smoke e não funcionais no Q4. Componente no Q1.

5.2 Gerenciamento de Risco

Risco é um evento/situação possível cuja ocorrência causa um efeito adverso. O gerenciamento de risco aumenta a chance de atingir objetivos, melhora a qualidade e a confiança dos stakeholders.

As atividades de gerenciamento de risco se dividem em duas grandes partes:

- **Análise de risco** = identificação + avaliação de risco.
- **Controle de risco** = mitigação + monitoramento de risco.

Quando as atividades de teste são selecionadas, priorizadas e gerenciadas com base no risco, chamamos isso de **teste baseado em risco**.

5.2.1 Definição e atributos do risco

Todo risco tem dois fatores:

- **Probabilidade:** a chance de o risco ocorrer (maior que 0 e menor que 1).
- **Impacto (dano):** as consequências se ocorrer.

Nível de risco = Probabilidade × Impacto. Quanto maior o nível de risco, mais importante tratá-lo.

5.2.2 Riscos de Projeto x Riscos de Produto

Distinção que cai quase sempre:

	Risco de PROJETO	Risco de PRODUTO
Relaciona-se a	Gerenciamento e controle do PROJETO	Características de QUALIDADE do produto

	Risco de PROJETO	Risco de PRODUTO
Exemplos	Atrasos, estimativas ruins, falta de pessoal, conflitos, problemas com fornecedor, suporte de ferramenta insuficiente	Funcionalidade ausente/errada, cálculo incorreto, vulnerabilidade de segurança, performance ruim, UX ruim
Impacto se ocorre	Afeta cronograma, orçamento, escopo	Insatisfação do usuário, perda de receita/reputação, dano a terceiros, penalidades, até morte

Analogia — Construir um restaurante

Risco de PROJETO: o fornecedor da cozinha atrasar a entrega, faltar verba, a equipe pedir demissão. São riscos de TOCAR o projeto.

Risco de PRODUTO: a comida sair intragável, o salão ser inseguro, o cardápio ter erros de preço. São riscos do que será ENTREGUE ao cliente.

5.2.3 Análise de Risco do Produto

Objetivo: dar consciência do risco do produto para focar o esforço de teste e minimizar o risco residual. Idealmente começa cedo no SDLC.

Composta por:

- **Identificação de riscos:** gerar uma lista abrangente (brainstorming, workshops, entrevistas, diagramas de causa-efeito).
- **Avaliação de riscos:** categorizar, determinar probabilidade/impacto/nível, priorizar e propor formas de lidar.

A avaliação pode ser quantitativa (nível = probabilidade × impacto) ou qualitativa (usando uma matriz de risco).

A análise de risco do produto influencia o RIGOR e o ESCOPO dos testes. Os resultados ajudam a: determinar escopo, níveis e tipos de teste, técnicas e cobertura, estimar esforço, priorizar testes (achar defeitos críticos cedo) e decidir se outras atividades além de teste reduzem o risco.

5.2.4 Controle de Risco do Produto

Compreende as medidas em resposta aos riscos. Divide-se em:

- **Mitigação:** implementar as ações propostas na avaliação para reduzir o nível de risco.
- **Monitoramento:** garantir que a mitigação é eficaz, obter mais informação e identificar riscos emergentes.

Opções de resposta a um risco (não só testar!): mitigar via testes, ACEITAR o risco, TRANSFERIR o risco, ou ter um plano de CONTINGÊNCIA. Ações de mitigação VIA TESTE incluem: escolher testadores adequados ao risco, aplicar nível de independência adequado, fazer revisões/análise estática, usar

técnicas e cobertura adequadas, aplicar tipos de teste apropriados, e fazer teste dinâmico (incluindo regressão).

Cai na prova

Decore a estrutura em 2x2: Análise de risco = Identificação + Avaliação. Controle de risco = Mitigação + Monitoramento. E nível de risco = probabilidade × impacto. Respostas possíveis a um risco: mitigar, aceitar, transferir, contingência.

5.3 Monitoramento, Controle e Conclusão

- **Monitoramento de teste:** coleta informação para avaliar o progresso e medir se os critérios de saída foram atingidos.
- **Controle de teste:** usa essa informação para dar diretrizes e ações corretivas (ex.: repriorizar testes, ajustar cronograma, adicionar recursos).
- **Conclusão de teste:** coleta dados de atividades concluídas para consolidar experiência e testware. Ocorre em marcos (fim de nível, fim de iteração, release, cancelamento).

5.3.1 Métricas de teste

Reconheça as categorias (cai "qual NÃO é métrica de teste"):

- Métricas de progresso do projeto (conclusão de tarefas, uso de recursos, esforço).
- Métricas de progresso do teste (casos executados, aprovados/falhos, ambiente).
- Métricas de qualidade do produto (disponibilidade, tempo de resposta, MTBF).
- Métricas de defeitos (nº e prioridade, densidade, taxa de detecção).
- Métricas de risco (nível de risco residual).
- Métricas de cobertura (requisitos, código).
- Métricas de custo.

5.3.2 Relatórios de teste

Relatório	Quando	Para quê
Relatório de PROGRESSO de teste	Durante o teste, periódico (diário/semanal)	Apoiar o controle contínuo; permitir ajustes
Relatório de CONCLUSÃO de teste	No fim (de um nível/projeto/iteração)	Resumir o estágio; alimentar testes futuros

Conteúdo do relatório de PROGRESSO: período, progresso (adiantado/atrasado), impedimentos e soluções, métricas, riscos novos/alterados, testes planejados para o próximo período.

Conteúdo do relatório de CONCLUSÃO: resumo, avaliação da qualidade vs. plano original, desvios do plano, impedimentos/soluções, métricas, riscos não mitigados e defeitos não corrigidos, lições aprendidas.

Públicos diferentes querem informações diferentes — isso influencia formalidade e frequência. Relatórios entre a própria equipe tendem a ser informais e frequentes; o relatório final segue modelo definido e ocorre uma vez.

5.3.3 Comunicação do status dos testes

Meios possíveis: comunicação verbal, painéis (dashboards de CI/CD, quadros de tarefas, burn-down), canais eletrônicos (e-mail, chat), documentação online e relatórios formais. Comunicação mais formal é apropriada para equipes distribuídas (distância geográfica/fuso).

5.4 Gerenciamento de Configuração (CM)

CM oferece a disciplina para IDENTIFICAR, CONTROLAR e RASTREAR produtos de trabalho (planos, casos, scripts, resultados, logs, relatórios) como itens de configuração.

Pontos que caem:

- Para itens complexos (ex.: ambiente de teste), o CM registra os componentes, relações e versões.
- Item aprovado para teste vira uma **baseline (linha de base)**, e só muda por processo formal de controle de mudanças.
- É possível reverter para uma baseline anterior para reproduzir resultados de testes antigos.
- Garante que todos os itens sejam identificados unicamente, versionados, rastreados e referenciados sem ambiguidade.
- No DevOps, o CM costuma ser automatizado dentro do pipeline.

5.5 Gerenciamento de Defeitos (K3 — preparar relatório)

Como achar defeitos é objetivo central do teste, um processo de gerenciamento de defeitos é essencial. Cuidado com o vocabulário: o que se relata inicialmente é uma ANOMALIA — que pode virar um defeito real OU outra coisa (falso positivo, change request). Isso é decidido durante o tratamento.

O fluxo típico: registrar → analisar → classificar → decidir a resposta (corrigir ou manter) → fechar. Defeitos de teste estático (especialmente análise estática) devem ser tratados de forma semelhante.

Objetivos de um relatório de defeitos:

- Dar informação suficiente para resolver o problema.
- Rastrear a qualidade do produto de trabalho.
- Dar ideias para melhorar o processo de desenvolvimento e teste.

Conteúdo de um relatório de defeitos (teste dinâmico) — itens que caem na K3:

- Identificador único.
- Título com resumo breve da anomalia.
- Data da observação, organização e autor (com cargo).

- Identificação do objeto de teste e do ambiente.
- Contexto do defeito (caso de teste, atividade, fase do SDLC, técnica/checklist/dados usados).
- Descrição da falha para reprodução (passos, logs, dumps, screenshots, gravações).
- Resultados esperados e resultados reais.
- **Severidade** (grau de impacto) e **prioridade** de correção.
- Status (aberto, adiado, duplicado, aguardando correção, aguardando reteste, reaberto, fechado, rejeitado).
- Referências (ex.: ao caso de teste).

Atenção / Pegadinha

Pegadinha de ouro: SEVERIDADE $\neq$ PRIORIDADE.

SEVERIDADE = o quão GRAVE é o impacto técnico do defeito (ex.: o sistema trava). É uma propriedade do defeito.

PRIORIDADE = o quão URGENTE é corrigir, do ponto de vista de negócio (ex.: corrigir agora ou na próxima release). É uma decisão de negócio.

Um defeito pode ter ALTA severidade e BAIXA prioridade (uma tela que ninguém usa trava) — ou o inverso (um erro de digitação no logo da empresa: baixa severidade, alta prioridade).

Capítulo 6 — Ferramentas de Teste

Peso na prova: apenas 2 questões (5%). É o menor capítulo. Foque em benefícios e riscos da automação — é o que mais cai.

6.1 Suporte de Ferramentas para Testes

Ferramentas apoiam muitas atividades. Reconheça as categorias (cai "associe a ferramenta à atividade"):

- Ferramentas de gerenciamento (do SDLC, requisitos, testes, defeitos, configuração).
- Ferramentas de teste estático (apoiam revisões e análise estática).
- Ferramentas de projeto e implementação (geram casos, dados, procedimentos).
- Ferramentas de execução e cobertura (executam testes automatizados, medem cobertura).
- Ferramentas de teste não funcional (fazem o que é difícil/impossível manualmente).
- Ferramentas de DevOps (pipeline, build, CI/CD).
- Ferramentas de colaboração (comunicação).
- Ferramentas de escalabilidade/implantação (máquinas virtuais, contêineres).

Atenção / Pegadinha

Frase curiosa do syllabus que pode aparecer: até uma PLANILHA é uma ferramenta de teste, no contexto do teste. Ou seja, ferramenta de teste é qualquer coisa que auxilie a testar — não precisa ser uma ferramenta sofisticada.

6.2 Benefícios e Riscos da Automação

Princípio fundamental: comprar uma ferramenta NÃO garante sucesso. Toda ferramenta exige esforço (introdução, manutenção, treinamento) para dar benefício real e duradouro.

Benefícios possíveis da automação:

- Economia de tempo reduzindo trabalho manual repetitivo (ex.: regressão).
- Prevenção de erros humanos simples (mais consistência e repetibilidade).
- Avaliação mais objetiva (ex.: cobertura) e métricas difíceis de obter manualmente.
- Acesso mais fácil a informação de teste (estatísticas, gráficos).
- Redução do tempo de execução → detecção mais cedo e menor time-to-market.
- Mais tempo para o testador criar testes novos e mais profundos.

Riscos possíveis da automação:

- Expectativas irreais sobre a ferramenta.
- Estimativas imprecisas do tempo/custo/esforço de introduzir e manter.
- Usar a ferramenta quando o teste MANUAL seria mais apropriado.
- Confiar demais na ferramenta (ignorar o pensamento crítico humano).
- Dependência do fornecedor (pode fechar, aposentar a ferramenta, dar suporte ruim).
- Software de código aberto que pode ser abandonado.
- Ferramenta incompatível com a plataforma de desenvolvimento.
- Escolher ferramenta que não cumpre requisitos normativos/segurança.

**Cai na prova**

A questão mais comum do Cap. 6 pede para distinguir benefício de risco, ou identificar um que NÃO pertence à lista. Lembre-se da ideia central: ferramenta não é mágica — exige esforço e pensamento crítico humano continua necessário.

Estratégia final para o dia da prova

Antes da prova: o checklist de revisão

Se você consegue responder estas perguntas de cor, está pronto:

1. Quais são os 7 princípios de teste e o que cada um significa?
2. Qual a diferença entre erro, defeito, falha e causa-raiz?
3. Verificação x validação; QA x QC; teste x depuração — sei distinguir?
4. Quais são os 5 níveis e os 4 tipos de teste? Alfa x Beta?
5. Confirmação x regressão; estático x dinâmico — claro na cabeça?
6. Os tipos de revisão e quem lidera cada um? Os papéis na revisão?
7. Sei DERIVAR casos com EP, BVA (2 e 3 valores), tabela de decisão e transição de estado?
8. Cobertura de instrução x ramificação (qual subsume qual)?
9. Os 3 Cs e o INVEST das histórias de usuário?
10. A fórmula da estimativa de três pontos: $E = (o + 4m + p)/6$?
11. Risco de projeto x produto; análise (identif.+avaliação) x controle (mitigação+monitoramento)?
12. Severidade x prioridade; DoR x DoD?
13. Benefícios x riscos da automação?

Durante a prova: táticas que ganham pontos

- **Gerencie o tempo:** 60 min / 40 questões = 1,5 min por questão. Não trave. Marque as difíceis e volte depois.
- **Nunca deixe em branco:** não há desconto por erro. Na dúvida final, chute — você tem 25% a 50% de chance.
- **Leia a pergunta até o fim:** atenção a palavras como NÃO, EXCETO, SEMPRE, NUNCA, MELHOR, PRIMEIRO. Elas mudam tudo.
- **Cuidado com absolutos:** alternativas com "sempre", "nunca", "todos", "garante" costumam estar erradas (lembre: teste não PROVA ausência de defeitos).
- **Elimine as obviamente erradas:** reduzir de 4 para 2 alternativas dobra sua chance.
- **Nas questões K3 (cálculo):** faça no rascunho. LISTE os valores/partições/colunas antes de contar; remova duplicatas (ex.: BVA).
- **Responda pelo SYLLABUS, não pela sua prática:** se a resposta 'do mundo real' diverge da definição oficial, a prova quer a oficial.

Os 12 erros que mais reprovam candidatos

1. Confundir verificação (especificação) com validação (necessidade).

2. Trocar QA (preventivo/processo) com QC (corretivo/produto).
3. Inverter a cadeia erro → defeito → falha.
4. Misturar os princípios 1 (presença, não ausência) e 7 (falácia da ausência).
5. Trocar Alfa (ambiente do dev) com Beta (ambiente do usuário).
6. Achar que a análise responde 'como testar' (é 'o que testar'; modelagem é 'como').
7. Esquecer de contar as partições INVÁLIDAS no EP.
8. Confundir BVA de 2 e de 3 valores.
9. Dizer que 100% de instrução garante 100% de ramificação (é o contrário).
10. Trocar severidade (impacto técnico) com prioridade (urgência de negócio).
11. Trocar DoR (entrada/Ready) com DoD (saída/Done).
12. Achar que o autor pode ser líder/relator numa inspeção (não pode).

Você já tem a prática. Com a teoria deste curso bem fixada e os simulados treinados, a aprovação é uma consequência natural. Bons estudos e boa prova!